

Program synthesis is the task of automatically finding programs that meet user intents that are described by correctness *specifications*, such as input-output examples and logical formulas. Program synthesis is now used successfully in software engineering, computer-aided education, end-user programming, and data cleaning [4, 2, 5, 14, 17, 12, 16]. For example, Facebook developed a tool that uses program synthesis to automatically fix bugs in production code [15].

Although program synthesis promises to increase the productivity of programmers by automating time-consuming tasks and to lower the bar of end-user programming, synthesizers are not as popular as other technologies such as SMT solvers. One reason people don't use synthesizers is that specifications are always ambiguous (even a formal specification of function properties is still ambiguous—there can be equivalent programs with different sizes or performance), and synthesizers often return solutions users don't want, e.g., the solution is an overfitted lookup table when the specification is given as input-output examples. Writing a full formal and less ambiguous specification can ease such pain, but is error-prone and not always straightforward, especially for end-users. Another issue is that most existing synthesis techniques do not tolerate noise in the specification—a small change in specifications could lead to a totally different solution. The goal of this proposal is to overcome these shortcomings by building a unified framework that is both able to tolerate noise and to allow users to specify preferences among different solutions, both in terms of syntactic and semantic criteria.*

Goal: Design a program synthesis framework that is predictable, user-friendly, and tolerant to noise.

Research Objectives. During the fellowship, my main research objectives are to:

1. Design a synthesis framework that supports semantic quantitative objectives, and hence can tolerate mistakes in user-given specifications. Designing an algorithm for problems in this framework will require methods to efficiently prune the search space according to their semantics, such that we don't need to evaluate all candidate solutions and check if they satisfy the given objectives.
2. Develop a multi-objective synthesis framework allowing both syntactic quantitative objectives on the program (e.g., should not use conditionals) as well as semantic quantitative objectives (e.g., complexity should be linear, or at least 90% of the specification should be satisfied). Solving problems in this framework will require new algorithms that can navigate the complex search space induced by the combination of quantitative objectives.
3. Design a user-friendly specification language that allows users to omit corner cases in the specification. Here, the challenge is how to find the right balance between the expressiveness of specifications, which allows users to easily write specifications, and limitations on specifications, which keep the problem amenable to synthesis techniques.

Intellectual Merits Little prior work exists on synthesis with quantitative objectives or allowing users to specify their preference, and the proposed work will introduce a new general formalization for expressing synthesis problems with semantic quantitative objectives and new algorithms for solving problems in such a framework. The proposed work will result in user-friendly specification languages leveraging syntactic and semantic constraints that allow users to provide simpler specification, along with syntactic quantitative objectives for reducing ambiguity; new algorithms for solving synthesis problem using such specifications. The proposed work will lead to new approaches to synthesize programs for noisy specification, and new algorithms to synthesize programs with partial specifications that omit corner cases.

Broader Impacts We will build open-source synthesis tools to improve productivity of programmers and to democratize programming problems. We hope our tools have the same impact as SMT solvers in the industry. Our techniques can be applied to personalized education. Our prior work [10] showed program repair could help students learn coding by telling them where the bugs are and how to fix them. With the proposed multi-objective synthesis framework, program repair problems can be seen as a specific instance where there are hard constraints in the form of test cases, and a solution needs to satisfy soft constraints encode similarity to the original buggy program.

*This work fits within the vision of the CCF Software and Hardware Foundations NSF directorate.

Candidate’s Plan

A synthesis framework that supports semantic quantitative objectives (Year 1). In our prior work [9], we showed how to add syntactic quantitative objectives (e.g, restricting the number of times an operator can be used) to synthesis problems. Instead, semantic quantitative objectives (e.g., identifying solutions that satisfy *most* of the specification) will make synthesizers tolerant to mistakes in the specifications.

Example 1. Consider a scenario where an end-user is using a synthesizer to write a regular expression [13]. The user provides a set of positive examples $\{12:14, 06:4, 07:03\}$ and a set of negative examples $\{07:4, 7:04\}$ for synthesizing a time-format expression accepting strings of the form $XX:YY$. Here, the user has made a mistake and the string $06:4$ is a false positive.

In the mentor’s prior work [6], they have shown how to solve synthesis problems with noisy input-output examples. We propose to define more general semantic quantitative objectives for specifications beyond input-output examples, e.g., logical formulas. We propose to specify our objectives using logics with weights [3]. An example of such logic is the MaxSMT problem, where a solution should satisfy a set of formulas whose total weight is at least W —the objective specified by the user. The problem of finding such a solution could be attacked by 1) enumerating solutions and evaluating their weights, 2) enumerating conjunctions of subformulas satisfying the objective and solving sub-problems with the conjunctions as specification and without quantitative objectives, or 3) sampling some concrete inputs for each subformula and solving the sub-problems with input-output examples. One challenge is to prove that a solution is optimal with respect to the given objectives. We plan to build on our recent work [9, 7, 8] on reducing the problem of proving optimality to an unrealizability problem—i.e., proving that a problem has no solution—and present new algorithms that are tailored to semantic objectives.

Milestone 1, 09/21: A tool for handling quantitative variations of the following benchmarks: 1) programming by noisy examples [6], 2) regular expressions with noisy examples [13], and 3) SyGuS benchmarks with hand-injected mistakes. We will then move to applications such as data-wrangling and super-optimization.

A multi-objective synthesis framework (Year 1/2). Specifications associated with semantic quantitative objectives are *soft* and ambiguous—there can be multiple solutions satisfying different parts of the specifications. We propose to allow multiple objectives to empower users to mitigate this problem.

Example 2. For the problem in Example 1, if we assign weight 1 to each positive example and ask for a solution with weight at least 2 (i.e., the user can make one mistake), there will be two possible solutions $e_1 := [0-9](2) : [0-9](2)$ which satisfies only two true positives, and $e_2 := ([0-9](2) : [0-9](2)) \mid (06 : 4)$ which satisfies all three positives. The syntactic constraint $|e^*| \leq 15$ will rule out the overfitted solution e_2 .

We propose to develop a multi-objective synthesis framework that supports 1) soft specifications as semantic quantitative objectives, 2) hard specifications, and 3) syntactic quantitative objectives. We will solve problems in such a framework using type-guided synthesis with refinement types introduced in the mentor’s prior work [11], which can contain both syntactic and semantic information. We will also investigate compositional approaches that solve individual objectives separately and combine the obtained solutions.

Milestone 2, 05/22: We will apply this framework to the benchmarks in the first milestone to obtain less ambiguous specifications and obtain better solutions than the ones returned by existing tools.

A user-friendly specification language (Year 2). Sometimes users know the main functionality of the expected program, but find it hard to write a formal specification covering all the corner cases. We plan to allow the user to provide a partial specification that constrains only parts of the inputs.

Example 3. A user wants to synthesize the absolute value function, but writing the full logical specification $\varphi_{abs} = (x > 0 \rightarrow f(x) = x) \wedge (x < 0 \rightarrow f(x) = -x)$ is hard for an end-user. We will allow the user to provide a partial specification “ $f(x) = x$ on infinite number of inputs” along with input-output examples $f(-1) = 1, f(-2) = 2$ and a syntactic objective that minimizes the size of the solution. This specification is enough for synthesizing `abs` and is composed of many natural easy-to-write individual objectives.

In REGEL [1], users can specify synthesis problems with natural languages and examples, which are then solved with a combination of learning and symbolic techniques. We propose to attack the problem in Example 3 using a similar approach.

Milestone 3, 12/22: To construct the natural language specifications, we will first collect benchmarks from SyGuS competition and then crowdsource English specifications. We will combine the natural languages specifications, hard specifications, and quantitative objectives into new benchmarks. By the end of this fellowship, we plan to solve these new benchmarks where the solutions we found are unambiguously correct.

References

- [1] Q. Chen, X. Wang, X. Ye, G. Durrett, and I. Dillig. Multi-modal synthesis of regular expressions. *arXiv preprint arXiv:1908.03316*, 2019.
- [2] L. D’Antoni, R. Samanta, and R. Singh. Qclose: Program repair with quantitative objectives. In *International Conference on Computer Aided Verification*, pages 383–401. Springer, 2016.
- [3] D. Dubois, L. Godo, and H. Prade. Weighted logics for artificial intelligence—an introductory discussion, 2014.
- [4] S. Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.
- [5] B. Hempel and R. Chugh. Semi-automated svg programming via direct manipulation. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, pages 379–390, 2016.
- [6] N. P. Hila Peleg. Perfect is the enemy of good: Best-effort program synthesis. *Europe’s longest-standing annual Programming Languages (ECOOP)*, 14(5):1222, 2020.
- [7] Q. Hu, J. Breck, J. Cyphert, L. D’Antoni, and T. Reps. Proving unrealizability for syntax-guided synthesis. In *International Conference on Computer Aided Verification*, pages 335–352. Springer, 2019.
- [8] Q. Hu, J. Cyphert, L. D’Antoni, and T. Reps. Exact and approximate methods for proving unrealizability of syntax-guided synthesis problems. *arXiv preprint arXiv:2004.00878*, 2020.
- [9] Q. Hu and L. D’Antoni. Syntax-guided synthesis with quantitative syntactic objectives. In *International Conference on Computer Aided Verification*, pages 386–403. Springer, 2018.
- [10] Q. Hu, R. Samanta, R. Singh, and L. D’Antoni. Direct manipulation for imperative programs. In *International Static Analysis Symposium*, pages 347–367. Springer, 2019.
- [11] T. Knoth, D. Wang, N. Polikarpova, and J. Hoffmann. Resource-guided program synthesis. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 253–268, 2019.
- [12] M. Kuperstein, M. Vechev, and E. Yahav. Automatic inference of memory fences. *ACM SIGACT News*, 43(2):108–123, 2012.
- [13] R. Pan, Q. Hu, G. Xu, and L. D’Antoni. Automatic repair of regular expressions. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–29, 2019.
- [14] O. Polozov and S. Gulwani. Flashmeta: a framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pages 107–126, 2015.
- [15] A. Scott, J. Bader, and S. Chandra. Getafix: Learning to fix bugs automatically. *CoRR*, abs/1902.06111, 2019.
- [16] M. Vechev, E. Yahav, and G. Yorsh. Abstraction-guided synthesis of synchronization. In *Proceedings of the 37th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 327–338, 2010.
- [17] H. Zhang, A. Jain, G. Khandelwal, C. Kaushik, S. Ge, and W. Hu. Bing developer assistant: improving developer productivity by recommending sample code. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 956–961, 2016.